IBM Machine Learning Professional Certificate

Deep Learning and Reinforcement Learning - Course Project

A Project Report

on

Comparative Analysis of Deep Learning Techniques for Image Classification and Generation on CIFAR-10 Dataset

by

Aryan Agarwal

July, 2026

Table of Contents

1. Introduction and Summary of the Dataset

This project explores a range of deep learning techniques for image classification and generation, using a single, consistent dataset throughout. The goal was not just to build one model, but to compare several different approaches side by side – from a simple baseline model to more advanced convolutional networks and generative models – in order to understand how each technique affects performance, efficiency, and the quality of learned representations.

1.1 Objectives

The project was designed around the following goals:

- Establish a simple baseline model to serve as a performance floor.
- Explore whether compressing images into a smaller representation (using autoencoders) helps or hurts classification performance compared to using raw pixel data.
- Compare multiple convolutional neural network designs to understand how architectural choices affect accuracy, model size, and speed.
- Use a generative model to create synthetic training images and test whether this kind of data augmentation improves classifier performance.
- Evaluate every model using the same set of metrics, so that a fair, direct comparison could be made at the end of the project.

1.2 Dataset Description

The dataset used throughout this project is CIFAR-10, a widely used benchmark dataset in computer vision. It consists of 60,000 small colour images, each 32x32 pixels, evenly split across 10 object categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

Of these, 50,000 images are designated for training and 10,000 for testing. Each class is equally represented, meaning there is no class imbalance to account for. The images are small and low-resolution by modern standards, which keeps computational requirements manageable, while still

presenting a genuinely challenging classification problem due to variation in pose, lighting, and background clutter within each class.

1.3 Why This Dataset

CIFAR-10 was chosen because it strikes a good balance between simplicity and realism. It is small enough to train multiple models on a personal computer in a reasonable amount of time, yet complex enough that simple models (like a basic MLP) clearly underperform compared to more sophisticated architectures – making it well suited to a comparative study like this one.

2. Data Preprocessing

2.1 Loading and Organizing the Data

The dataset was loaded using a standard deep learning library and consolidated into a single structured table containing each image, its numeric label, its human-readable class name, and whether it belonged to the training or test split. This made it easy to inspect, filter, and verify the data before any modeling began.

2.2 Verification Checks

Before proceeding, basic checks were carried out to confirm the data was loaded correctly:

- Confirming the total number of images and the train/test split sizes matched expectations.
- Checking that all 10 classes were equally represented in the training set.
- Visually inspecting one sample image per class to confirm that labels were correctly assigned, and verifying image shape and datatype.

```
Total rows: 60000
split
train    50000
test     10000
Name: count, dtype: int64

Class balance (train):
label_name
frog          5000
truck         5000
deer          5000
automobile    5000
bird          5000
horse         5000
ship          5000
cat           5000
dog           5000
airplane      5000
Name: count, dtype: int64

Single image shape/dtype: (32, 32, 3) uint8
```

2.3 Normalization

Raw image pixel values range from 0 to 255. Rather than using this raw scale, the pixel values were normalized using the average and spread (mean and standard deviation) of the training data. This is a standard step that helps neural networks train faster and more reliably, since it keeps input values in a small, consistent range rather than large raw numbers.

2.4 Train/Validation/Test Split

While the dataset naturally comes with a train/test split, a portion of the training data (5,000 images) was further set aside as a validation set. This validation set was used throughout the project to monitor model performance during training and to decide when to stop training (to avoid overfitting), while the test set was reserved purely for final, unbiased evaluation.

2.5 Data Augmentation

For the convolutional models later in the project, simple data augmentation was applied – small random flips and shifts of the training images. This helps models generalize better by exposing them to slightly varied versions of the same image, reducing the risk of memorizing specific training examples. This augmentation was only applied during training, never during evaluation.

2.6 Preparing Different Data Formats

Since different models in this project require different input shapes, two versions of the data were prepared:

- A "flattened" version, where each image is converted into a single long list of pixel values, used for the baseline MLP model.
- The original image format (height x width x color channels), used for all convolutional models and the generative model.

3. Baseline Multi-layer Perceptron

3.1 Purpose of the Baseline Model

Before moving to more advanced architectures, a simple Multi-Layer Perceptron (MLP) was built to serve as a baseline. An MLP is one of the most basic types of neural networks – it consists of a stack of fully connected layers, where every input is connected to every neuron in the next layer. Unlike convolutional networks, it has no built-in understanding of spatial structure (i.e., it doesn't inherently know that neighboring pixels are related), which makes it a useful, simple reference point to compare more sophisticated models against.

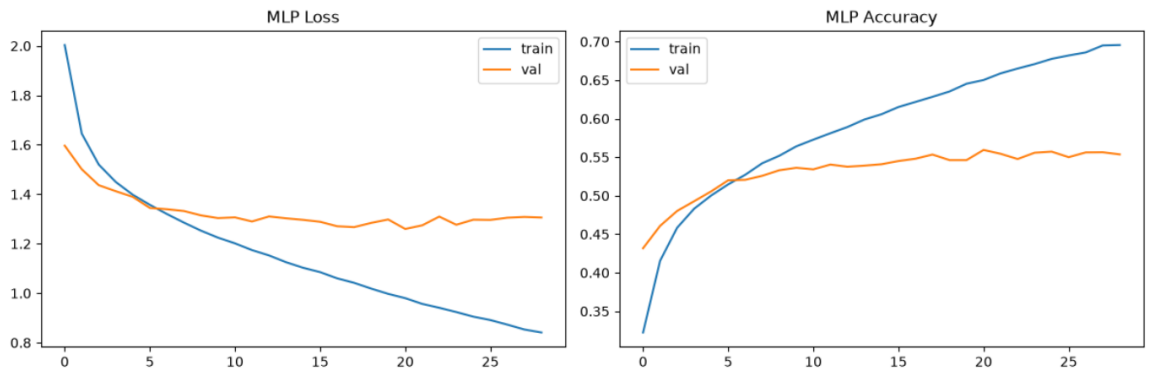
3.2 Model Architecture

The baseline model takes the flattened image (all pixel values in a single row) as input and passes it through three fully connected Dense layers of decreasing size (1024 -> 512 -> 256), with each layer followed by a BatchNormalization layer and a Dropout layer, to prevent overfitting. The final Dense layer outputs a probability for each of the 10 classes. The total parameters of the final model was over 3.8 million.

3.3 Training Process

The model was trained using a standard 'sparse_categorical_crossentropy' loss function and an adaptive Adam optimizer with a learning rate of 0.001, with training automatically stopped once performance on the validation set stopped improving, to avoid wasting time or overfitting.

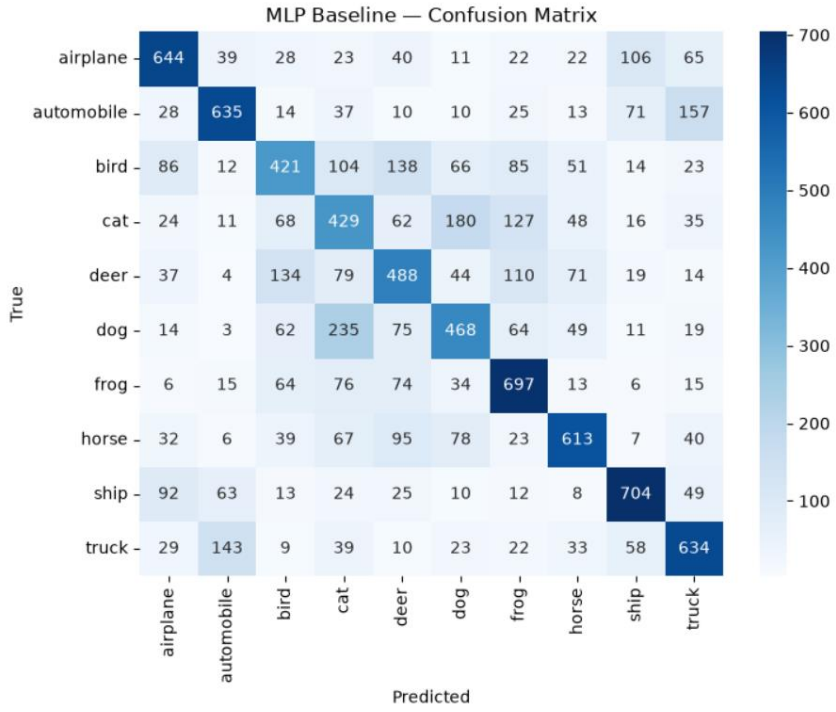
The loss and accuracy curves obtained were as follows:



3.4 Results

The baseline MLP achieved a test accuracy of approximately 57%, which serves as the performance floor for the rest of the project. This is noticeably lower than any of the convolutional models discussed later, which is expected because the MLP has no way of recognizing visual patterns like edges or shapes that don't depend on exact pixel position.

The confusion matrix for baseline MLP is shown below:



3.5 Observations

The confusion matrix reveals that the model struggles the most with visually similar classes (for example, cats vs. dogs, or automobiles vs. trucks), which is a common pattern across all models in this project and reflects genuine visual ambiguity in the dataset rather than a flaw specific to this model.

4. Autoencoders

4.1 Purpose

This section explores whether images can be compressed into a much smaller representation without losing the information needed to classify them correctly. Two types of models were used for this: a standard Autoencoder and a Variational Autoencoder (VAE). Both are designed to compress an image down to a small set of numbers (called the "latent representation") and then reconstruct the original image from that compressed form.

4.2 Autoencoder – How It Works

An autoencoder consists of two parts: an encoder, which compresses the image into a small vector of numbers, and a decoder, which attempts to reconstruct the original image from that compressed vector. The model is trained purely to make its reconstructions look as close to the original image as possible – it is never told anything about class labels during this training.

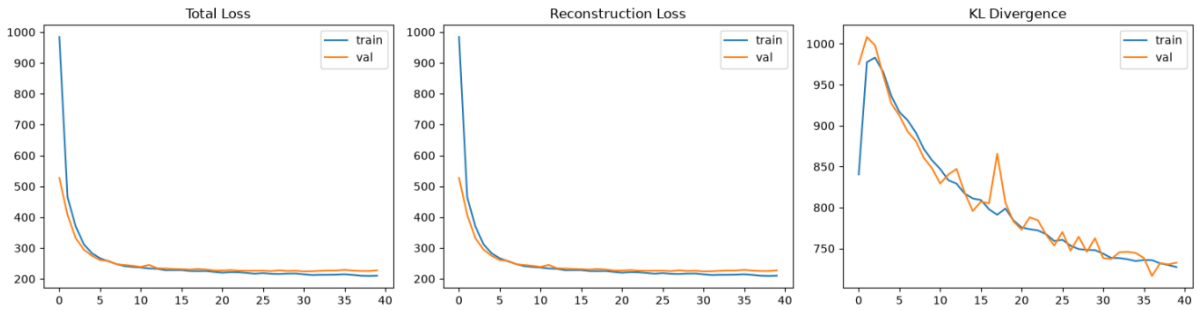
In this project, the images were compressed from over 3,000 numbers (32x32x3 pixels) down to just 128 numbers. The autoencoder consisted of 3 convolutional Conv2D layers followed by a Flatten layer, while the decoder was built using Dense and Reshape layer, followed by 3 Conv2DTranspose layers. Then some test reconstructions were run to check model accuracy.

4.3 Variational Autoencoder – How It Works

A Variational Autoencoder (VAE) works similarly, but with one key difference: instead of compressing an image into a single fixed set of numbers, it compresses it into a small range of possible values (a distribution), which introduces some randomness into the process. This tends to produce a

smoother, more organized compressed space, which is useful for generative tasks, though it can sometimes produce slightly blurrier reconstructions than a plain autoencoder.

For the VAE, a custom training step, test step and loss function was written. The encoder and decoder architecture remained unchanged. After training the model for 40 epochs, the loss curves obtained were as follows:



4.4 Using Compressed Features for Classification

To test whether these compressed representations retain useful information, the same baseline MLP architecture from Section 3 was retrained, but this time using the 128-number compressed representation (from both the autoencoder and the VAE) instead of the raw pixel values.

First, the VAE latent features were extracted by running .predict() on the train, validation, and test sets using just the encoder portion of both the AE and the VAE, following which the latent space was used to train the MLP.

4.5 Results and Comparison

Model	Test Accuracy
MLP on raw pixels	~57%
MLP on autoencoder features	~57%
MLP on VAE features	~56%

The results did not vary much from the baseline, and infact slightly reduced for the MLP trained on the VAE latent space.

4.6 Observations

Interestingly, compressing the images down to a much smaller size did not significantly hurt (or help) classification accuracy when paired with the same simple MLP. This suggests that most of the information needed for basic classification can be preserved in a much smaller representation, but also confirms that compression alone doesn't add classification power; the bottleneck was never optimized to separate classes, only to reconstruct images faithfully.

5. Convolutional Neural Networks

5.1 Why Convolutional Networks

Convolutional Neural Networks (CNNs) are specifically designed for image data. Unlike the multilayer perceptrons used upto this point, which treat every pixel independently, CNNs use small filters that scan across the image, allowing the model to use spatial context to recognize patterns like edges, textures, and shapes regardless of where they appear in the image. This makes CNNs far better suited to image classification tasks.

5.2 Overview of the Three Variants

To understand the different design choices affect performance, 3 different CNN architectures were built and trained under identical conditions (same data, same optimizer, same training schedule):

Variant 1 – Shallow CNN

- A simple network with just two shallow layers followed by a small fully connected layer. This serves as a CNN-specific baseline.
- Consisted of two Conv2D + MaxPooling2D layers, and one Dense layer, for a total of 540 thousand parameters

Variant 2 – Deep Residual CNN:

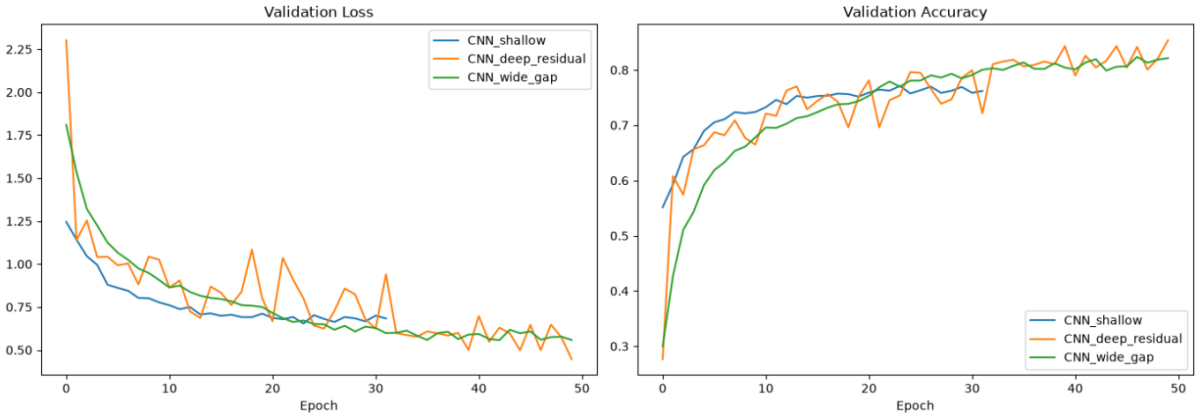
- A deeper network that uses "residual connections", which are essentially shortcuts that allow information to skip over layers. This makes it possible to train deeper networks without them becoming harder to optimize, and typically improves accuracy at the cost of more computing power and longer training times.
- Consisted of over 35 layers, with repeating residual “blocks” of SeparableConv2D and BatchNorm layers, separated by MaxPooling2D layers after each block, and a couple of dense layers at the very end.
- Had a total of almost 600,000 parameters.

Variant 3 – Wide CNN with Global Average Pooling

- A network with more filters per layer (making it "wider") but a lighter final stage, using a technique called global average pooling instead of a large fully connected layer, which significantly reduces the total number of parameters.
- Consisted of blocks of SeparableConv2D layers separated by MaxPooling2D layers, followed by a GlobalAveragePooling2D layer, with only one Dense layer at the end.
- Had a total of 170,000+ parameters.

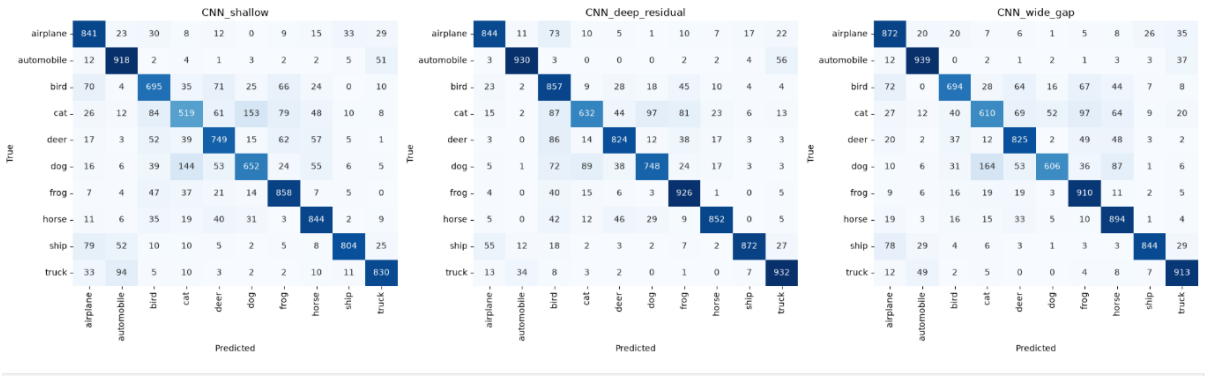
5.3 Training Process

All three models were trained using the same Sequential data augmentation, Adam optimizer, and early stopping strategy described in Section 2, ensuring that any differences in performance are due to architecture rather than training setup.



5.4 Results

Model	Test Accuracy	Parameters
Shallow CNN	~77%	~545K
Deep Residual CNN	~84%	~594K
Wide CNN (GAP)	~81%	~170K



5.5 Observations

The deep residual network achieved the highest accuracy, confirming that added depth and skip connections help the model learn more effectively. Notably, the wide CNN with global average pooling achieved strong accuracy with far fewer parameters than the other two models, demonstrating that architectural efficiency (not just size) plays a major role in performance. All three CNNs substantially outperformed the MLP-based models from Sections 3 and 4, confirming the advantage of using architectures designed specifically for image data.

6. Conditional Generative Adversarial Networks

6.1 Purpose

This final modelling section explores whether artificially generated images can be used to improve classifier performance – a technique known as data augmentation via generative modelling. A Conditional Generative Adversarial Network (CGAN) was built for this purpose.

6.2 How a CGAN Works

A GAN consists of two competing models trained together:

- A **Generator**, which starts from random noise and attempts to create realistic images.
- A **Discriminator**, which attempts to distinguish real images from the generator's fake images.

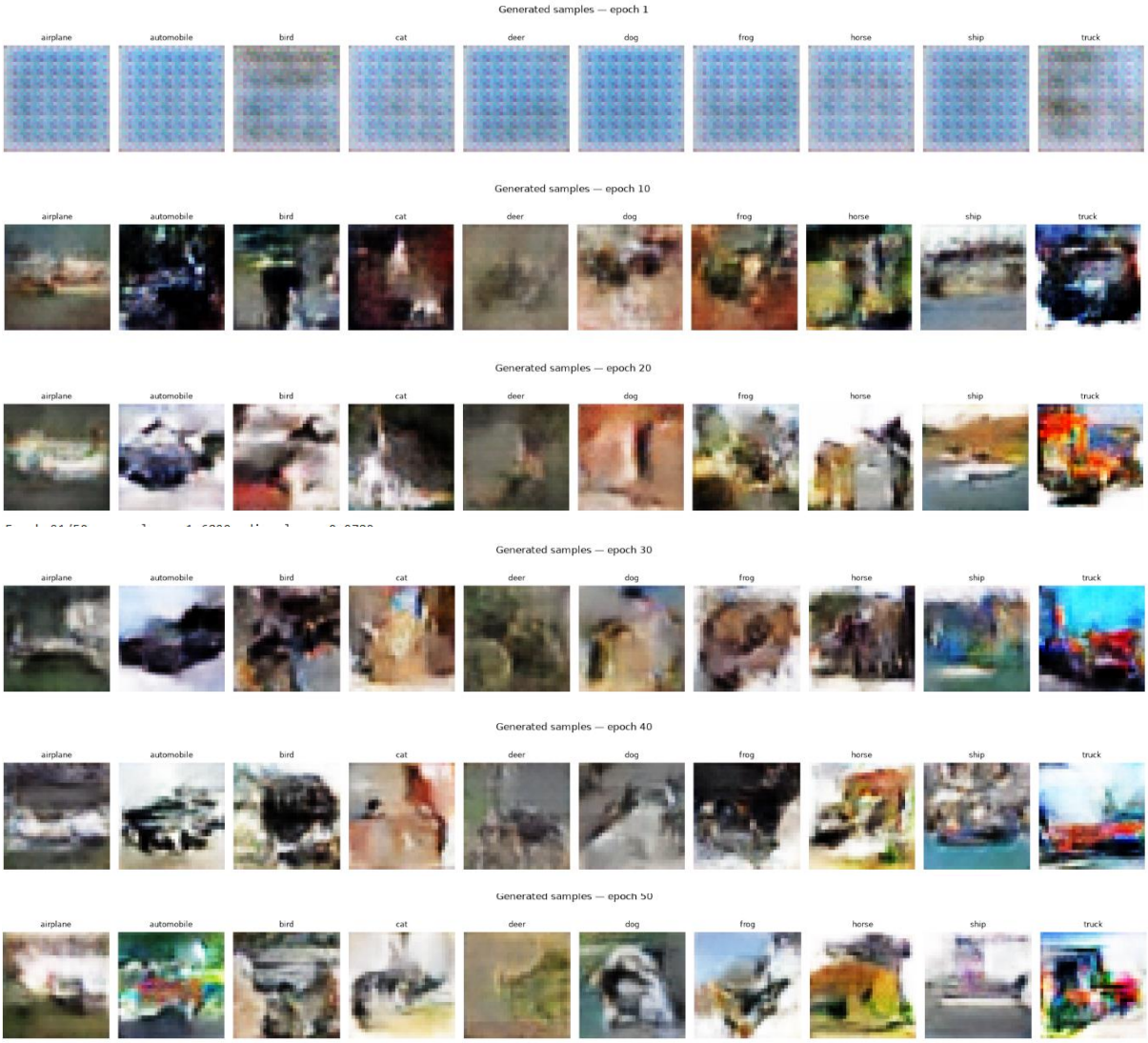
These two models are trained in competition – the generator tries to fool the discriminator, while the discriminator tries not to be fooled. Over time, this pushes the generator to produce increasingly realistic images. The "conditional" part means that both the generator and discriminator are also given the class label as input, allowing the generator to be told which specific class of image to create (e.g., "generate a cat" rather than just "generate something").

6.3 Training Process

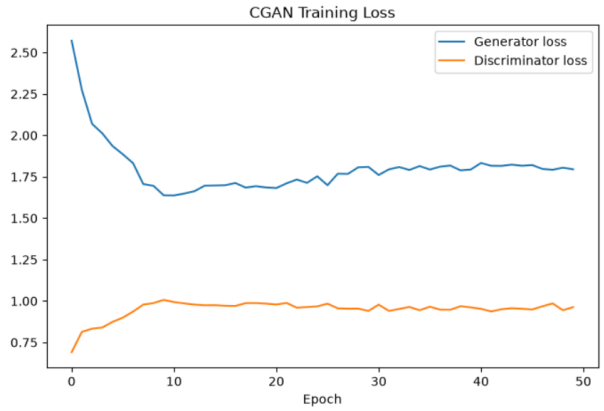
The generator architecture consisted of multiple blocks of a Conv2DTranspose layer along with BatchNormalization and LeakyReLU layers. Each layer upsamples the input deconvolutions to gradually expand the dimensions of the output to 32x32x3, the dimension of our CIFAR-10 images. This model consisted of over a million parameters.

The discriminator did the exact opposite, downsampling the input from the generator through multiple convolutional blocks of Conv2D + BatchNorm + LeakyReLU + Dropout layers, to finally output a one-hot vector using a Dense layer, consisting of the predicted class. This part of the model consisted of a little over 675 thousand parameters.

The generator and discriminator were trained together over many epochs. Generated sample images were periodically saved during training to track how image quality improved (or didn't) over time.



As can be seen from the above results, while the model did initially improve for the first 10 or so epochs, the performance plateaued afterward, and even seemed to slightly worsen, as echoed by the loss curves.



6.4 Generating Synthetic Training Data

Once trained, the generator was used to create a batch of new, synthetic images for each class. These synthetic images were combined with the real training data to create an "augmented" training set, larger and more varied than the original. A total of 10,000 synthetic images of equal class distribution were added to the training set, bringing the total train set size to 55,000.

6.5 Testing the Augmentation

To test whether this synthetic data actually helped, the shallow CNN architecture from Section 5 was retrained from scratch, once on the original real data only, and once on the combined real + synthetic dataset. Both versions were evaluated on the same, untouched real test set, ensuring a fair comparison.

6.6 Results

Model	Test Accuracy
Shallow CNN (real data only)	~77%
Shallow CNN (real + synthetic data)	~75%

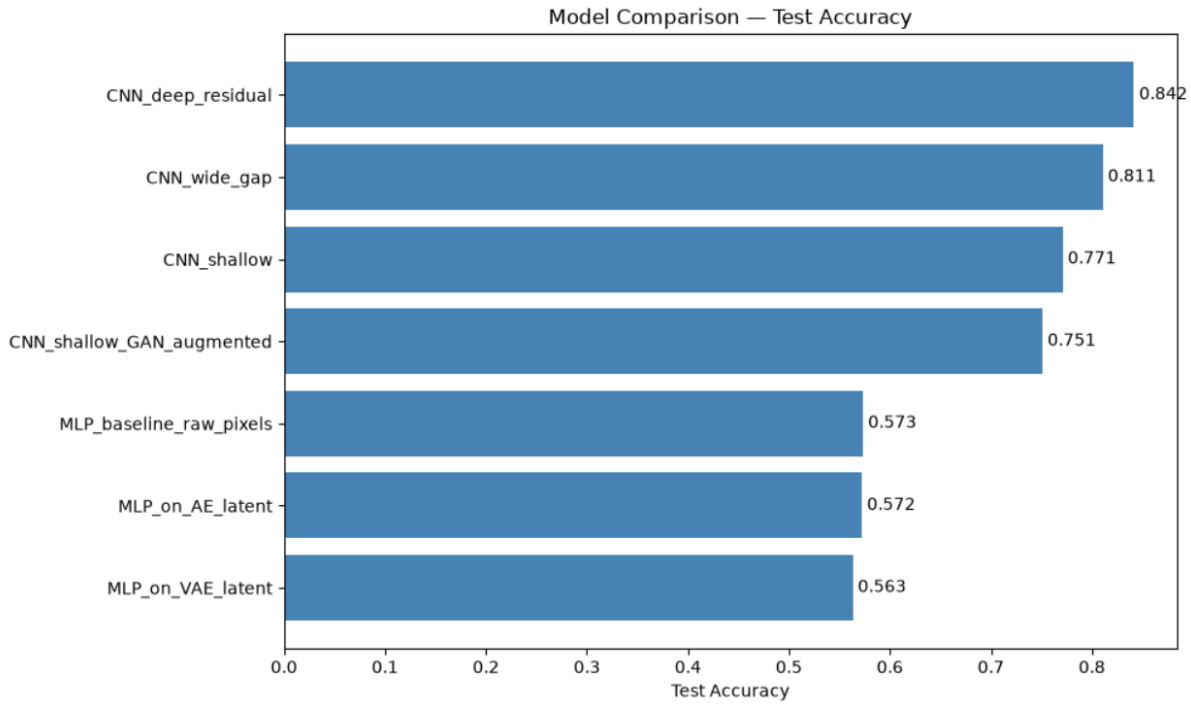
6.7 Observations

Contrary to the initial expectation, adding GAN-generated images slightly reduced classifier accuracy rather than improving it. This is a meaningful and realistic finding rather than an error: generative models like this typically require substantial training time and tuning to produce images that are both realistic and clearly representative of their intended class. If the generated images are not yet high quality or occasionally ambiguous between classes, they can introduce noise into training rather than useful variety. This highlights that the quality of generated data matters more than the quantity when it comes to augmentation.

7. Key Findings and Insights

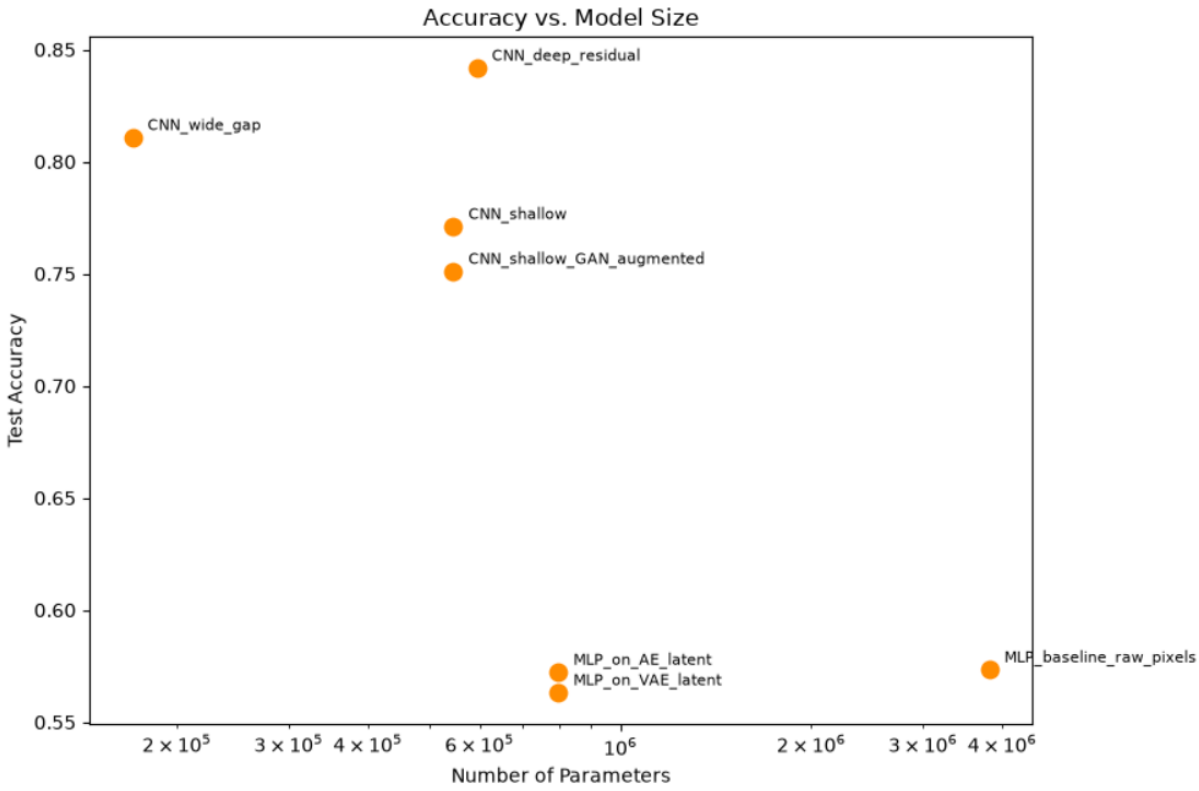
7.1 Model Performance Summary

	model_name	accuracy	macro_f1	precision_macro	recall_macro	n_params	inference_time_sec
0	MLP_baseline_raw_pixels	0.5733	0.572900	0.574883	0.5733	3812618	2.806372
1	MLP_on_AE_latent	0.5722	0.569160	0.570180	0.5722	797962	2.698686
2	MLP_on_VAE_latent	0.5631	0.559285	0.561311	0.5631	797962	2.719216
3	CNN_shallow	0.7710	0.768328	0.769513	0.7710	545098	4.596532
4	CNN_deep_residual	0.8417	0.841436	0.847566	0.8417	593578	20.774675
5	CNN_wide_gap	0.8107	0.807262	0.814207	0.8107	170122	20.899694
6	CNN_shallow_GAN_augmented	0.7510	0.746624	0.752014	0.7510	545098	4.048687



7.2 Architecture Matters More Than Raw Model Size

The comparison across CNN variants showed that a well-designed, efficient architecture (the wide CNN with global average pooling) could match or approach the performance of a larger model while using significantly fewer parameters. Meanwhile, the plain MLP had the largest parameter count of any model in the project, yet the weakest performance – a clear illustration that parameter count alone is not a reliable predictor of accuracy.



7.3 Compression Preserves Class Information Reasonably Well

Compressing images to a fraction of their original size using an autoencoder or VAE did not meaningfully harm classification performance when using a simple classifier. This suggests these compressed representations retain much of the information relevant to distinguishing between classes, even though they were never explicitly trained with classification in mind.

7.4 Depth and Residual Connections Improve Performance

Among the CNN variants, the deepest model with residual connections achieved the best accuracy, reinforcing a well-established idea in deep learning: additional depth, when paired with techniques like residual connections, allows models to learn more complex patterns without becoming harder to train.

However, the wide-gap model performed almost as good as the deep-residual model, despite having a parameter size of only ~30% of the latter, suggesting that similar performance could be achieved by using a more efficient architecture.

7.5 Generative Data Augmentation Is Not Automatically Beneficial

Perhaps the most important finding of this project is that adding GAN-generated images to the training set slightly hurt performance rather than helping it. This challenges the common assumption that "more data is always better" and instead points to the importance of generated data quality; a poorly converged generator can introduce misleading examples that confuse rather than help a classifier, regardless of the amount of images added. In fact, adding a lot of low-quality generated images may even impact performance to a notable extent.

7.6 Consistent Evaluation Enabled Fair Comparison

By evaluating every model using the same metrics (accuracy, F1 score, parameter count, inference time) and always testing on the same untouched real test set, this project was able to make direct, fair comparisons across very different model types – from simple MLPs to deep CNNs to GAN-augmented classifiers.

8. Conclusions

8.1 Summary

This project set out to explore a full spectrum of deep learning techniques on a single dataset, from the simplest baseline model to more advanced convolutional and generative approaches. Across seven trained classifiers, a clear performance hierarchy emerged: convolutional architectures substantially outperformed MLP-based approaches, with the deep residual CNN achieving the best overall accuracy. Compression techniques (autoencoders and VAEs) proved capable of retaining most class-relevant information despite drastically reducing the size of the input data, while the generative augmentation experiment produced a valuable, if counterintuitive, negative result, underscoring that synthetic data quality, not just quantity, determines whether augmentation helps.

8.2 Practical Takeaways

- For image classification tasks, even a simple CNN reliably outperforms a much larger MLP, confirming the practical value of architectures suited to the structure of image data.
- Efficient architectural choices (such as global average pooling) can reduce model size substantially with only a modest accuracy trade-off, which is valuable in resource-constrained settings.
- Generative data augmentation should be validated empirically rather than assumed to help – a poorly-converged generator can be counterproductive.

8.3 Limitations

This project was intentionally scoped to run on a personal computer, which placed practical limits on training time, model size, and the number of training epochs - particularly for the CGAN, which typically requires significantly longer training to produce consistently high-quality, class-accurate images. Additionally, all models were trained and evaluated using a single random seed/run for each configuration, meaning some of the smaller performance differences observed between models could be partly attributable to natural training variability rather than purely architectural differences.

8.4 Future Steps and Potential Improvements

- **Longer, more stable GAN training**: Extending the CGAN's training time considerably, and considering more advanced training techniques designed to stabilize GAN training, would likely produce higher-quality generated images and could reverse the negative augmentation result seen in this project.
- **Tuning the augmentation ratio**: Rather than using a single fixed ratio of synthetic to real images, testing multiple ratios (e.g., 10%, 25%, 50% synthetic) would help identify whether a smaller, more careful dose of synthetic data might help even if a large one does not.
- **Filtering generated images by quality**: Using the discriminator itself, or a separately trained classifier, to filter out low-confidence or ambiguous generated images before adding them to the training set could improve the quality of the augmented dataset.
- **Repeating experiments across multiple runs**: Running each model several times with different random seeds and averaging the results would help distinguish genuine architectural effects from run-to-run variability.
- **Exploring additional architectures**: Testing more modern CNN designs, or exploring whether the compressed autoencoder/VAE features could be improved by training the compression and classification steps together (rather than separately), could further improve results.
- **Expanding evaluation metrics**: Incorporating additional measures of generated image quality (beyond visual inspection) would provide a more quantitative way to track and improve the CGAN's output over time.